

SEAL

SECURITY FRAMEWORKS

v0.1

A pocket 101 of the
SEAL frameworks

feat. **THE RED GUILD**

Security Alliance

SECURITY FRAMEWORKS

v0.1

A pocket 101 of the
SEAL frameworks

Security Alliance

Table of Contents

About this book	7
How to use this	10
Operational security	13
Physical security	16
Privacy	19
Community	22
Insider threats	25
Wallet security	28
Protocol multisig.....	31
Treasury	34
DevSecOps	37
Supply chain	40
Infrastructure	43

Monitoring	46
AI security	49
Incident management	52
Safe Harbor	55
Testing and reviews	58
Awareness	61
If it is happening now	64
Conclusion	68

ABOUT – SECURITY ALLIANCE

Security Alliance (SEAL) is a nonprofit organization dedicated to securing the future of cryptocurrency. Under the leadership of whitehat hacker **samczsun**, the organization designs security projects and coordinates across the crypto industry so that useful practices actually get used.

Learn more at securityalliance.org

ABOUT – THE RED GUILD

A non-profit organization, think-tank-alike, devoted to do security as a public good within the Ethereum ecosystem. Its work spans security research, responsible disclosure, education, open-source tooling, community awareness and more. It is best known for developing gamified learning experiences like **Damn Vulnerable DeFi**, and their recent platform the **Phishing Dojo**.

Learn more at theredguild.org

blog.theredguild.org · phishingdojo.com

About this book

This mini-book is a print map of the **Security Frameworks**, an initiative by the **Security Alliance**. The site is the living document. This edition is rewritten for a pocket you can hand to a founder, a signer, or a new hire on day one.

It is not every page. It is the 101 of the frameworks that are actually built: takeaway, a few insights, the first moves. Dev-gated stubs, tool catalogs, and unfinished sections stay on the website. Empty chapters make a worse companion than a short one.

Four other pocket books already exist. Coercion lives in Physical Security. Travel lives in OpSec While Traveling. Protocol signers live in Protocol Multisig. Self-custody lives in Wallet Security. This book will not pretend to be those.

This print cut changes order, cuts website navigation, and turns the same controls into reading you can do on a train. It does not invent new controls to fill gaps.

Help us improve

The live frameworks move. If a control here is wrong, incomplete, or now dangerous in your jurisdiction, fix

it upstream so the site and the next printing match.

`github.com/
security-alliance/frameworks`

If you value this kind of resource, you can support the work at our website.

`securityalliance.org/donate`

Human safety comes first

No asset, key, policy, or evidence goal is worth more harm. If someone is being coerced, put this book down and open the Physical Security companion.

How to use this

The frameworks exist because the same losses keep happening: a seed photographed, a Discord admin hijacked, a dependency swapped, a signer who clicked the first hash they saw. Cryptography did not fail. The rest of the system did.

Read this as a map, not as a certificate. Each chapter is one framework. If a chapter names a dedicated companion, that companion is the deep cut. If it names the site, go there before you copy a control into production.

Disclaimer

This is not legal advice, and it is not a recovery service. It will not match every chain, team, or threat model. Some dollar figures and thresholds in the source are examples, not defaults.

The contents are free on the Security Frameworks site. Suggest changes there.

How this book is organized

1. **People.** OpSec, physical, privacy, community, hiring.
2. **Keys.** Wallet, protocol multisig, treasury.

3. **Ship.** DevSecOps, supply chain, infra, monitoring, AI.
4. **Break.** Incident, Safe Harbor, testing and reviews.
5. **Keep going.** Awareness. Then cards for the moment you do not want to reread a section.

Operational security

OpSec is a cycle. Name what would hurt if it leaked. Name who wants it. Find the holes. Apply controls that match the risk, then do it again. A policy binder nobody runs is not OpSec.

This is the everyday layer: devices, browsers, passwords, MFA, travel. Cryptography does not save a laptop left unlocked in an airport lounge.

Insights

- Layered controls. One lock failing should not dump the keys.
- Least privilege. Accounts get the access the job needs, then it gets taken back.
- Isolate work. Signing does not happen in the same browser that reads Discord.
- Web3 is public. The chain shows what you did. Your chat should not.

Quick wins

1. Hardware key for MFA. SMS recovery is a SIM-swap waiting.

2. Password manager. Unique secrets.
No shared root accounts in chat.
3. Separate browser or OS profile for signing.
4. Before a trip, read the OpSec While Traveling companion.

The full OpSec framework is on the site.
Travel is its own pocket book.

Physical security

Physical security protects people and assets from threats that bypass cryptography: coercion, theft, surveillance, tampering. OpSec hides information. This domain stops what a body can be forced to do.

A hardware wallet is useless if the person holding it is in a room with a wrench. Life first. Then keys.

Coercion and duress is the mature core. Facility, counter-surveillance, and hardware provenance pages are still in progress on the site. This book will not fake them.

Insights

- One person should not be able to move the pile.
- Delay beats panic. Time-locks and extra signers exist so a kidnapping is not an instant drain.
- Families, homes, and routines are part of the threat model. Attackers map those first.
- If it is happening now, compliance is not the goal. Survival is.

Quick wins

1. Know who can spend if you disappear for a day.

2. Do not keep the seed where you sleep if that is also where you are found.
3. Practice a duress path before you need one.
4. Hand the Physical Security companion to anyone who holds a key worth coercing.

Privacy

Privacy is security of personal and team data against unnecessary collection. Shrink the footprint first. Encryption and network tools then reduce what is left.

Leaks feed phishing, physical targeting, and reconnaissance. A founder photo from a conference plus an on-chain label is enough to start a wrench story.

Insights

- Active trail and passive trail are both real. Posts, WHOIS, data bro-

kers, old Git commits.

- A VPN is not anonymity. It hides your path from the cafe. It does not hide you from the site you log into.
- Encrypted chat still has metadata. The member list is often the prize.
- Product claims change. Verify a tool before you bet a key on it.

Quick wins

1. Search your name and the project. Remove what you can from brokers.
2. HTTPS everywhere. Do not type seeds or keys on public Wi-Fi, VPN or not.

3. Separate identities for public posting and for signing.
4. Encrypted messenger with a recovery story you actually tested.

Community

Community channels are high-trust broadcast paths. A compromised operator account turns that trust into phishing at scale. The fake claim link in Discord has stolen more user funds than most novel contract bugs.

Users believe the official server. That is the point. Harden the people who can post there.

Insights

- Admin roles are keys. Treat them like keys.

- Bots and OAuth apps are extra admins with worse memory.
- Telegram recovery is a phone number unless you add two-step verification.
- Federation and invite links copy rooms to machines you do not control.

Quick wins

1. MFA on every official account. No SMS if you can avoid it.
2. Two people to post announcements. One compromised intern should not drain the users.

3. Pin the only official URLs. Tell people search results lie.
4. Revoke stale OAuth and old admin sessions this week.

Platform pages on the site cover Discord, X, Telegram, Matrix, Session, and SimpleX.

Insider threats

DPRK IT workers are fraudulently hired remote staff who fund a sanctioned regime. They can also steal, extort, or sabotage the supply chain. Hiring and access control are the primary defenses, not antivirus after the fact.

This is not a rare edge case. It is a hiring-process failure mode with legal and theft consequences.

Insights

- Fake identity is the job. Interviews, laptops, and stand-ins are part of the method.
- Shared credentials and laptop-from-home setups are how one hire becomes the whole repo.
- Paying the wrong person can be a sanctions problem, not only an HR problem.
- After discovery, assume they mapped internals. Rotate, do not just fire.

Quick wins

1. Video interviews with identity that matches payroll. Refuse stand-ins.
2. Company-owned machines. No personal laptops on production.
3. Least privilege from day one. No standing admin for contractors.
4. If a hire already feels off, read the DPRK pages and the incident playbook before you ping them in Slack.

Wallet security

Wallet security is control of keys and verification of every signature. Hardware and multisig fail if you sign attacker-controlled calldata because a website told you to.

Never sign blindly. The device screen is the transaction.

This chapter is the 101. The Wallet Security companion is the book you hand to the person who signs.

Insights

- Custody is who can spend. An exchange login is still a custodian.
- Hot is for spend. Cold is for savings. Mixing them is how a drain becomes a life event.
- The seed is the wallet. A photo of it is a copy of every account it will ever derive.
- Approvals are spending permission, not a login.

Quick wins

1. Know who can spend, including the backup holder you forgot.

2. Buy hardware direct. Reject a device that arrives already set up.
3. Write the seed offline. Never type it into a computer.
4. Official URL, matching address, matching amount, hardware screen agrees. Otherwise reject.

Protocol multisig

If you read one multisig page, make it this: timelocks, function separation, dedicated keys, calldata verification, and a monitored queue beat unenforced policy.

A 4-of-7 with no delay and one shared seed is one compromised person away from a governance drain. The Protocol Multisig companion is the signer-first deep cut. Safe is the default stack there.

Insights

- Separate upgrade, pause, parameters, and treasury. One Safe that does everything is a single point of failure.
- Emergency paths should pause, not upgrade or move funds.
- Hash checks confirm you signed the intended bytes. Decode still tells you what those bytes do.
- Anyone can add you to a fake Safe that looks like yours. Bookmarks, not the dashboard.

Quick wins

1. Inventory every privileged role and which multisig holds it.
2. Hardware wallets for every signer. Dedicated address per Safe.
3. Delegated proposer. Sign only. A non-signer executes.
4. Alert on queued transactions, including a channel the internal team cannot mute.

Treasury

Treat custodial treasury accounts like cash vaults. Classify by impact and how fast you need the funds. Document who can move them. Verify every large transfer before it is irreversible.

This is for organizations that hold significant value through a custodian or co-managed setup. Day-to-day protocol governance is the multisig book.

Insights

- Classification sets the control bar. A critical account without a delay is a labeled vault with the door open.
- Registration is how you notice a silent permission change.
- Address poisoning and rushed wires work because people copy from chat.
- MPC and policy engines help. They do not replace a second human who checks the destination.

Quick wins

1. List every account, who can approve, and how fast it can move.
2. Whitelist destinations. Delay new ones.
3. Test transaction, then the real one.
Two people on a call for first-time recipients.
4. Never copy an address from explorer history.

DevSecOps

DevSecOps works when security is in the path from plan to ship. Automated checks stop unsafe changes before production. Sandboxes contain the blast when untrusted code or tooling runs.

If the pipeline can deploy, the pipeline is a production key. Treat it that way.

Insights

- Untrusted repos execute. Git hooks, IDE plugins, install scripts, agent

tools. Opening a folder is already a decision.

- Signed commits only help if you verify the signer.
- Secrets in CI logs are public secrets with extra steps.
- Governance proposals are code too. An upgrade text is a transaction waiting to happen.

Quick wins

1. Branch protection, MFA, CODEOWNERS. No force push to default.
2. SAST and tests on every change. Fail the build on high findings you have not waived.

3. Ephemeral runners. No standing SSH to the build box.
4. Do not run random tools on the machine that holds deploy keys. Sandbox them.

Supply chain

Your software is only as secure as its weakest dependency. Know what you depend on. Verify integrity. Have a plan for when something in the chain is compromised.

Attackers do not need your Solidity. They need the npm package your front end imports, the CDN that serves it, or the contractor who opened a PR.

Insights

- The Web3 chain is longer: compilers, wallet connectors, RPC, oracles,

bridges, hardware.

- Lockfiles are evidence. Regenerating them casually is how a swap hides.
- Typosquatting still works because people install from a README they skimmed.
- Human supply chain is the DPRK chapter. Hardware is the wallet chapter. Do not mix the runbooks.

Quick wins

1. Pin versions. Commit the lockfile. Diff it in review.
2. Inventory direct dependencies and who owns each one.

3. Restrict who can publish the front end. Integrity hashes if you can.
4. When a library is hijacked, assume users already loaded it. Incident, not a quiet bump.

Infrastructure

Web3 front ends and ops still run on centralized infrastructure. Inventory it. Harden cloud and DNS. Weigh one-vendor simplicity against a shared failure domain.

Smart contracts get the audits. DNS hijacks steal the users. Both are the protocol as far as the victim is concerned.

Insights

- You cannot protect what you have not listed.

- Registrar locks and DNSSEC are boring until the domain is gone.
- One vendor for host, DDoS, registrar, and DNS is easy. It is also one ticket away from a total outage.
- Zero trust means verify again, not trust the office IP.

Quick wins

1. List domains, clouds, and who can change them.
2. Registry lock on the main domain. 2FA on the registrar, not SMS.
3. Default-deny network. Patch the boxes that still exist.

4. Monitor DNS and cert changes like you monitor the vault.

Monitoring

Monitoring only improves security when every alert has an owner and a documented response. Detection without action is a diary of the hack.

This framework is on-chain: large transfers, mints, admin changes, upgrades, oracle drift. Host metrics belong in DevSecOps and infra.

Insights

- If nobody can pause, the alert is trivia.

- Thresholds that fire all day get muted. Tune them or lose them.
- One provider is one outage. Critical invariants deserve two independent watchers.
- External monitoring matters so an inside compromise cannot silence the bell.

Quick wins

1. Write, for each alert: who gets it, what they do, how they confirm it worked.
2. Watch admin, upgrade, and treasury events first.

3. Page a human. Slack-only is how nights are missed.
4. Tie alerts to the incident chapter before you buy another dashboard.

AI security

Agentic AI turns instruction-following into irreversible actions. Prefer run-time gates and isolation over a clever system prompt.

A model that can browse, run tools, or draft a transaction is a new intern with production access and no judgment. Prompt templates will not save you.

Insights

- Models treat text as instructions. Untrusted pages and tickets are control input.

- Block or quarantine unsafe outputs before they leave. Review after send is theater.
- Developer agents need sandboxes. Non-developer SaaS needs data classification and approved apps.
- Isolation for tool runners lives in DevSecOps. Implement it there.

Quick wins

1. No production keys in the agent environment.
2. Allow, warn, or block at the tool call, not in a later log.
3. Do not paste seeds, dump files, or customer data into a consumer

chat.

4. Treat AI-browser content as hostile until inspected.

Incident management

Decide who leads, how you communicate, and what to freeze before the incident. Web3 windows are short. Many losses are irreversible.

Plans written under stress lose to plans practiced in calm. SEAL 911 exists for when you need a war room. Have the number before you need the number.

Insights

- Pause paths and signer reachability are the first hour. Blog posts are not.
- Unconfirmed claims in public make the second incident.
- Playbooks on the site cover drainers, DPRK, malware, and front-end or DNS takeovers. Use them.
- Forensics needs logs you kept on purpose. You will not invent them at 3am.

Quick wins

1. Name an incident lead and a backup. Write the phone tree.

2. Know how to pause, and who can. Dry-run it.
3. Save SEAL 911 and your legal contact offline.
4. Afterward, write what you will do differently. Then change the system, not only the doc.

Safe Harbor

Safe Harbor lets a protocol pre-authorize whitehats to rescue funds during an active exploit, under published scope and rules. It is not a bug bounty, and it is not cover for the person who started the hack.

Without it, whitehats watch a drain they could have stopped because the lawyers have not answered yet. That is how Nomad sat open for hours.

Insights

- Only live or imminent exploits.
Disclosure before the hack is still a bounty problem.
- Whitehats return funds to your recovery addresses. Bounty terms are set in advance.
- The initiator who “rescues” with a second account is still a blackhat.
No bounty.
- Do not rewrite the legal standard.
Set scope: contracts, chains, recovery addresses, KYC, bounty cap.

Quick wins

1. Read the eligibility checklist on the site.
2. Publish recovery addresses you actually control.
3. Cap the Safe Harbor bounty at or below the existing bounty so faking a rescue does not pay.
4. Adopt on-chain. Uniswap and others already did.

`safeharbor.securityalliance.org`

Testing and reviews

No single test type proves a contract safe. Combine unit and integration coverage, fuzzing, static analysis, and, where the math is load-bearing, formal methods. Then see whether the suite notices real faults.

An external review is a time-boxed expert pass. It reduces risk and teaches the team. It is not a certificate of safety. Pair it with prep, fixes, and tests that keep running after the PDF.

Insights

- Coverage without assertions is a green bar that lies.
- Mutation testing asks whether your tests would catch a one-line cheat.
- Audits fail when the spec is missing and the scope is whatever is in the repo.
- Findings you do not fix are a press release, not a review.

Quick wins

1. Unit and fork tests on every change.
Fuzz the invariants that would bankrupt you.

2. Slither or equivalent in CI. Do not ignore high findings.
3. Write the spec the reviewers should attack. Freeze scope.
4. Track every finding to a fix or an explicit accept. Then keep testing.

Awareness

Awareness is recognizing risk signals early, especially unexpected requests and urgency tricks, so tools are not the only line of defense.

Culture fails when only scanners are expected to notice danger. A hurried “can you just sign this” from a known name is still an attack until you verify it on another channel.

Insights

- Pause is a control. Attackers buy speed.

- Out-of-band verification beats a perfect-looking email.
- Skills decay. Last year's phishing talk does not cover this week's drainer.
- Peer support. People miss things alone that a second person would catch.

Quick wins

1. No signing, payouts, or admin changes from a single chat message.
2. Call the person on a number you already had.
3. Recurring, short drills beat a yearly slideshow.

4. Reward the person who slowed a bad request down.

If it is happening now

These are prompts, not a local emergency plan. Read them before you need them.

Someone is being coerced

1. Life first. Keys later.
2. Open the Physical Security companion.
3. Do not coach them to be a hero over the phone.

4. After they are safe, assume every secret they could reach is burned.

About to sign

1. Official URL. Not a search result.
Not a chat link.
2. Function, destination, and amount match.
3. Hardware screen agrees. If it does not, reject.
4. Full Wallet Security companion if this is a habit, not a one-off.

Users are being drained

1. Pause if you can. Take the front end down if that is the vector.
2. Official channels only. Pin the real URL.
3. Incident lead, then SEAL 911 if you need a war room.
4. DNS, CDN, and admin keys are suspects until proven otherwise.

A hire or dependency feels wrong

1. Stop adding access. Do not confront in the same channel they control.

2. DPRK pages for people. Supply chain pages for packages.
3. Rotate what they could have touched.
4. Legal and sanctions questions are not optional if money already moved.

Conclusion

The frameworks are one place to put the controls that keep getting rediscovered after a loss. This book is the map. The site is the living document. The dedicated companions are the deep cuts.

You will not get this perfect. The job is to make a blind signature expensive, a hijacked admin noisy, and a coerced signer insufficient. Keep the authority small. Keep the humans alive.

Check the site before you treat a page here as current. Then go fix the one control you still do not have.

`frameworks.securityalliance.org`

ACKNOWLEDGMENTS

This pocket book follows public Security Alliance frameworks. Stewards named on the site as of this snapshot:

AI Security. curtis0x, masterfung, munamwasi

Community. NFTDreww

DevSecOps. Elliot

DNS. Raiders

Incident. EK, Nick K

Monitoring. Josep Bove

OpSec. Seth Hallem

Physical. AllyPry

Safe Harbor. Dickson Wu

Security Testing. Patrick Collins

Wallet. Piña (pinalikefruit)

Open frameworks in this cut still need a steward. See the site Spotlight Zone.

SOURCE SNAPSHOT

This cut follows reviewed public overviews of the Security Alliance frameworks. Web-site navigation, in-progress stubs, and dedicated companion runbooks are not in the book.

Branch develop

Commit cc80ea044d86

Commit date 2026-09-14

Build date 2026-09-14

Online frameworks.securityalliance.org

Framework authors in this snapshot

Security Alliance contributors named on the overviews this cut follows.

Check the site before you treat this as current.

@ 2026 Security Alliance / The Red Guild

Security Frameworks copyright 2023–2026 Security Alliance and contributors. This pocket edition is Creative Commons Attribution-ShareAlike 4.0 International.

creativecommons.org/licenses/by-sa/4.0/

Share and adapt, including commercially, if you give credit, say what changed, link the license, and keep adaptations under the same license.

Gentium Book is SIL OFL 1.1. Latin Modern is GUST Font License.

Pocket edition – 2026

Published by Security Alliance (SEAL) · The Red Guild